Representation and semantics

Arithmetic combines finite-width representations with operation rules. An adder processes bit patterns. Signed interpretation, instruction semantics, and status controls determine what happens when a mathematical result is outside the representable range.

The chapter develops integer ALUs, iterative multiply/divide hardware, and IEEE 754 floating point. Every model needs operand width, result width, rounding, exceptions, and timing assumptions.

Interpret `0xffffffff + 1` both ways before discussing its circuit.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The arithmetic problem

A fixed 32-bit word cannot represent every integer, fraction, or real number. Overflow, underflow, rounding, and special values are part of a representation contract.

Hardware trades latency, throughput, area, and precise exceptions. A multiplier can generate more partial products in parallel. A floating-point adder retains extra bits for rounding, which requires additional datapath and pipeline state.

Define the required result before choosing a gate diagram.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.1 The basic model

Modular integer instructions compute in the ring modulo 2^n. Trapping and saturating instructions impose different policies. Floating point maps an exact arithmetic result into a finite representable set F.

$$y = \mathrm{round}(x \circ z).$$

The rounding function, exceptional conditions, and special values are part of the operation. Correct rounding does not imply an infinitely precise mathematical result.

Integer division by zero depends on the ISA or language. IEEE floating-point `0/0` is invalid and ordinarily produces a quiet NaN when no trap is enabled.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

3.2 Two's-complement addition and subtraction

$$x = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i.$$

The low n result bits of an adder are the same for signed and unsigned addition. Whether they represent the mathematical signed sum depends on overflow.

$$a - b = a + \overline{b} + 1 \pmod{2^n}.$$

Complementing the second input and setting carry-in to one lets subtraction share an adder.

A correct modular bit pattern can still be an unrepresentable signed mathematical result.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deriving signed overflow

Let the operand signs be s_a, s_b and the result sign be s_r. Equal operand signs with a changed result sign indicate overflow.

$$ov = \overline{s_a \oplus s_b} \wedge (s_a \oplus s_r) = c_{\text{into MSB}} \oplus c_{\text{out of MSB}}.$$

Unsigned overflow depends only on carry out. `0x7fffffff+1` overflows signed addition. `0xffffffff+1` overflows unsigned addition while signed −1 plus 1 is valid.

Never replace both tests with an unqualified "overflow equals carry."

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Check carries separately from signed overflow

For an eight-bit adder, let c_7 enter the sign position and c_8 leave it. Signed overflow is $ov = c_7 \oplus c_8$.

Operation	Adder inputs	Result	c_7	c_8	Unsigned carry	Signed overflow
`0x7f+0x01`	`01111111+00000001`	`10000000`	1	0	0	1
`0x80+0xff`	`10000000+11111111`	`01111111`	0	1	1	1
`0x80-0x01`	`10000000+11111110+1`	`01111111`	0	1	1	1
`0xff+0x01`	`11111111+00000001`	`00000000`	1	1	1	0

The last row has a carry without signed overflow. `slt` and `sltu` also interpret identical bits differently. Subtraction uses $a + \bar{b} + 1$, with overflow $(s_a \oplus s_b) \wedge (s_a \oplus s_r)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Adder timing and exceptions

A ripple-carry adder propagates carry along the width. Lookahead and parallel-prefix designs shorten this dependency using more gates and wiring.

An overflowing `add` must not commit an ordinary destination result in place of its exception. `addu` retains the modular result without the signed-overflow trap.

Clock design uses the worst valid path and setup/hold constraints, not average operand behavior. Result generation and exception detection can have distinct paths.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Saturating multimedia arithmetic

Saturation clamps an out-of-range result to an endpoint. Signed eight-bit $100 + 100$ saturates to 127 instead of wrapping to a negative value.

This avoids wraparound but loses reversibility. It cannot replace modular arithmetic arbitrarily in checksums or cryptographic algorithms.

Packed SIMD arithmetic operates on several short lanes. Carries do not cross lane boundaries. Lane width and saturation policy belong to the instruction semantics.

Audio clipping and raw byte addition require different numerical contracts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.3 Multiplication hardware

An exact unsigned $n \times n$ product can require $2n$ bits. Two 32-bit inputs therefore need up to a 64-bit result.

$$AB = \sum_{k=0}^{n-1} b_k (A \ll k).$$

Shift-and-add accumulates a shifted multiplicand for each set multiplier bit. Shifting handles a power of two. Generating arbitrary partial products also requires bitwise gating.

Truncating an input or partial product before the exact computation changes the problem.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sequential multiplication and correctness

For unsigned n-bit M and Q, inspect Q0, conditionally compute $C : A \leftarrow A + M$, and shift the full (C, A, Q) right. C enters A's high bit. For M=13 and Q=11:

Round	Q0	Add result C:A	A after shift	Q after shift
0			`0000`	`1011`
1	1	`0:1101`	`0110`	`1101`
2	1	`1:0011`	`1001`	`1110`
3	0	`0:1001`	`0100`	`1111`
4	1	`1:0001`	`1000`	`1111`

The final product is `10001111` =143. Combined states R0..R4 are 11,109,158,79,143.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Sequential multiplication: prove the register invariant

Fix the initial multiplier $q < 2^n$, and define $R_k = 2^n A_k + Q_k$ after round k. The exact invariant is

$$R_k = \left\lfloor \frac{q}{2^k} \right\rfloor + M(q \bmod 2^k)2^{n-k}, \qquad 0 \le k \le n.$$

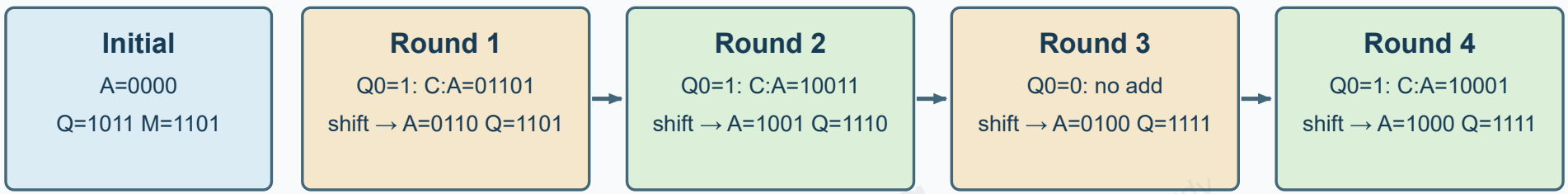
At $k = 0$, $A_0 = 0, Q_0 = q$, so it holds. When $k < n$, the second term is even. The current low bit is therefore original multiplier bit q_k. The conditional addition produces full combined value $R_k + q_k M 2^n$, including the carry. Shifting gives

$$R_{k+1} = \left\lfloor \frac{R_k + q_k M 2^n}{2} \right\rfloor = \left\lfloor \frac{q}{2^{k+1}} \right\rfloor + M(q \bmod 2^{k+1})2^{n-k-1}.$$

The equality uses $q \bmod 2^{k+1} = (q \bmod 2^k) + q_k 2^k$, completing induction. At $k = n$, $q < 2^n$ makes the first term zero, so $R_n = Mq$. A contains the high half and Q the low half. Carry C enters A during the shift and is not lost. The processed old Q0 leaves; old Q1 becomes the next decision bit.

Sequential multiplication: match the register movements

Four-bit shift-and-add states for 13 × 11



Final product: (A,Q)=10001111_2 = 143 = 0x008f.

C is the add carry. Shift (C,A,Q) right: C enters A, old Q0 leaves, and old Q1 is the next decision bit.

$$T_{\text{serial}} = t_{\text{setup}} + nt_{\text{iter}} + t_{\text{finalize}}.$$

Thirty-two iterations are 32 cycles only if each add and shift fits one cycle. Separate-cycle operations require a different cost. A parallel tree reduces dependent additions while adding wires, compressors, and power. Pipelining reduces initiation interval without necessarily reducing one operation's latency.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Signed multiplication starts with a width contract

The exact product of two n-bit two's-complement inputs fits in $2n$ signed bits. One method multiplies magnitudes and applies the sign at the end. Booth recoding instead generates signed partial products directly.

Do not take the absolute value of the most negative input in its original signed width. Four signed bits represent $[-8, 7]$: the positive magnitude of -8 needs five signed bits. An unsigned four-bit container can represent that magnitude as `1000`.

Here radix-2 Booth takes signed n-bit multiplicand M and multiplier Q. Internally, sign-extend M to $n + 1$ bits, use an $n + 1$-bit accumulator A, and retain one history bit q_{-1}.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

What do the two Booth bits decide?

Write the original multiplier as $q_{n-1} \cdots q_0$, append $q_{-1} = 0$, and define $d_i = q_{i-1} - q_i$. Read the pair with **the current low bit first, the history bit second**:

$Q_0\ q_{-1}$	d	Action before shifting
`00`	0	Keep A
`01`	+1	A ← A + M
`10`	−1	A ← A − M
`11`	0	Keep A

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why does the recoding preserve the signed value?

Collect the contributions of each original bit. A positive contribution comes from the next digit and a negative one from the current digit. The top bit has no following positive term, leaving the negative weight required by two's complement.

The recoding preserves the signed value:

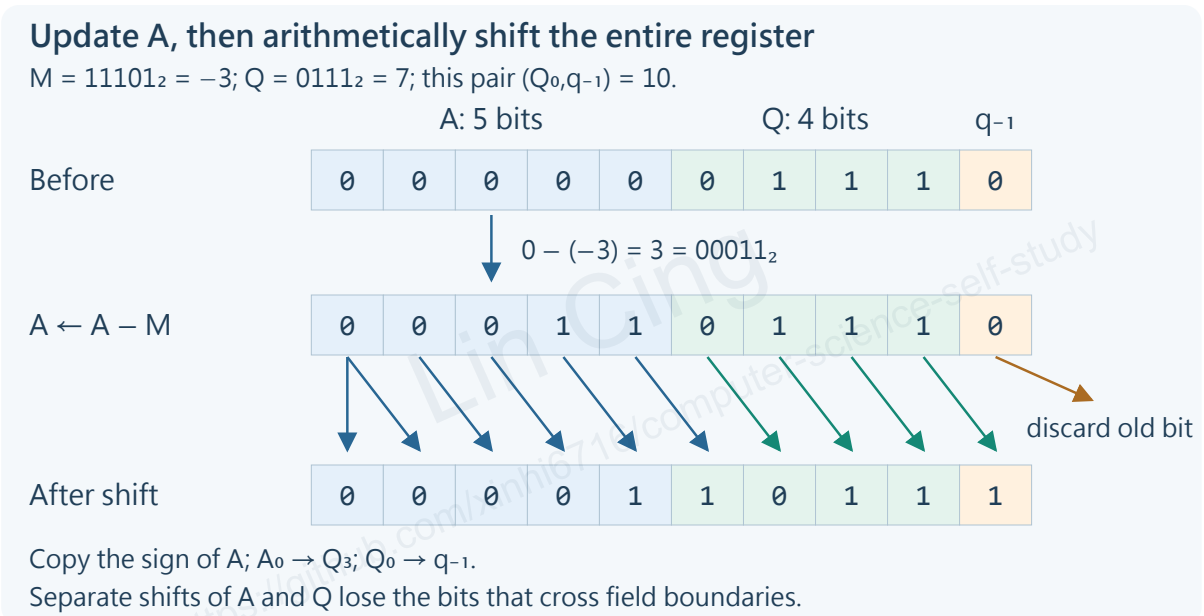
$$\begin{aligned}\sum_{i=0}^{n-1}(q_{i-1}-q_i)2^i &= \sum_{i=0}^{n-2} q_i 2^i - q_{n-1}2^{n-1} \\ &= Q_{\text{signed}}.\end{aligned}$$

Each intermediate bit contributes $(2^{i+1}-2^i)q_i = 2^i q_i$. The highest bit retains a negative weight, explaining why the rule also handles negative multipliers.

For `0111`, low-to-high digits are −1, 0, 0, +1, giving −1+8=7. For `1101`, they are −1, +1, −1, 0, giving −1+2−4=−3. Individual digits may be negative while their weighted sum equals the original multiplier.

Add or subtract, then shift the concatenated register

Initialize $A = 0$, load Q, and set $q_{-1} = 0$. Repeat n times: update A, then **arithmetically shift the entire (A, Q, q_{-1}) right by one bit**.



Copy A's sign bit; old A_0 enters Q's top bit and old Q_0 enters the history bit. After n rounds, drop the history and redundant top sign bit to obtain the 2n-bit product.

−3 times 7: trace all four rounds

Let $n = 4$, sign-extend $M = -3$ as `11101`, and start with `00000 0111 0`. Each row shifts only after its addition or subtraction; do not use the post-shift pair to select that same row's action.

Round	Old $Q_0 q_{-1}$	A action	After add/sub: A Q q_{-1}	After shift: A Q q_{-1}
1	`10`	−M	`00011 0111 0`	`00001 1011 1`
2	`11`	0	`00001 1011 1`	`00000 1101 1`
3	`11`	0	`00000 1101 1`	`00000 0110 1`
4	`01`	+M	`11101 0110 1`	`11110 1011 0`

Final $(A, Q) =$`11110 1011` is nine-bit -21. Removing the redundant sign bit gives `11101011`, still -21 in eight bits.

Original multiplier `0111` has low-to-high digits $(-1, 0, 0, +1)$: $7 = -1 + 8$ and $(-3) \cdot 7 = 3 - 24 = -21$. Only two rounds add or subtract, but all four rounds still shift.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

The extra sign bit covers the most negative input

For $M = -8, Q = -3$, the first pair is `10`, requiring $A - M = +8$. With only four bits for A, M, pattern `1000` instead means -8: the first round is already wrong.

$-8 \times (-3)$: widen before negating the multiplicand

4-bit A and M: fails	5-bit A and M: correct
M = 1000_2 = -8	M = 11000_2 = -8
$-$M = 1000_2 still means -8	$-$M = 01000_2 = $+8$
First A$-$M: 1000, not +8	First A$-$M: 01000 = +8
After shift: 1100 0110 1	After shift: 00100 0110 1
After four rounds: 11101000_2 = -24	After four rounds: 00011000_2 = $+24$

Both start with A=0, Q=1101_2, q_{-1}=0; only the A/adder width differs.

Five bits preserve +8 before shifting to +4; the four-bit path shifts −8 to −4. The next page proves the extra bit is sufficient for every input.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Prove the accumulator width is sufficient

The extra bit has a range proof. After each shift, $A \in [-2^{n-1}, 2^{n-1} - 1]$. Adding or subtracting the n-bit M stays within $[-2^n, 2^n - 1]$, which $n + 1$ bits represent exactly. Shifting restores the earlier range. This model does not rely on accidental cancellation of intermediate overflow.

Initially A=0 satisfies the bound. If A starts a round in that range, A+M ranges from -2^n to $2^n - 2$; A−M ranges from $-2^n + 1$ to $2^n - 1$. Keeping A unchanged also fits in the same n+1-bit range.

The next arithmetic shift replaces A by the floor of half the updated value. This restores $[-2^{n-1}, 2^{n-1} - 1]$. Induction proves that every addition or subtraction is exact, including the minimum-negative multiplicand.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Express the register state as an invariant

Let u be the **unsigned value** of the original multiplier bits. Digits $d_i = q_{i-1} - q_i$ still refer to those original bits. After k rounds define

$$R_k = 2^n \operatorname{signed}(A_k) + \operatorname{unsigned}(Q_k), \quad D_k = \sum_{i=0}^{k-1} d_i 2^i.$$

The invariant is

$$R_k = \left\lfloor \frac{u}{2^k} \right\rfloor + M 2^{n-k} D_k, \quad 0 \le k \le n.$$

Initially $R_0 = u, D_0 = 0$. For $k < n$, the low $n - k$ bits of Q_k have not been overwritten by incoming accumulator bits. Its low bit is original q_k, and the history bit is q_{k-1}. Thus the hardware pair really selects d_k.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

One round preserves the invariant and yields the product

Updating A adds $d_k M 2^n$ to R_k. The concatenated arithmetic shift gives

$$R_{k+1} = \left\lfloor \frac{R_k + d_k M 2^n}{2} \right\rfloor .$$

Substitute the invariant. For $k < n$, the term containing M is divisible by two, even when negative. Therefore

$$R_{k+1} = \left\lfloor \frac{u}{2^{k+1}} \right\rfloor + M 2^{n-k-1}(D_k + d_k 2^k).$$

This is the next round's invariant. After n rounds, $0 \le u < 2^n$ makes the first term zero. The recoding identity gives

$$R_n = M D_n = M Q_{\text{signed}}.$$

An arithmetic right shift rounds a negative integer toward negative infinity, not toward zero. The floor in the recurrence accounts for this distinction.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Fewer additions need not mean fewer cycles

`0111` needs arithmetic only at the ends of its run of ones. For `0101`, every pair is `10` or `01`, requiring four additions/subtractions; unsigned shift-add needs only two additions. Radix-2 Booth is not cheaper for every input.

In this model each round still controls and shifts the register, so both inputs take n rounds. Converting arithmetic counts into latency or throughput needs a timing contract: zero-run skipping, combined add/shift cycles, and pipelining are separate choices. Two fewer additions do not establish a twofold speedup.

Radix-4 processes two new bits plus one overlapping history bit. It changes partial-product count and selection circuitry; changing only the diagram's shift distance does not specify a radix-4 implementation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Radix-4 signed-digit table

For even n, read $(q_{2j+1}, q_{2j}, q_{2j-1})$, with $q_{-1} = 0$. Adjacent groups overlap by one bit; sign-extend odd widths first. Equal digits share a row below; all eight patterns are included.

Three-bit patterns	Digit c_j	Partial-product multiple
`000`, `111`	0	0
`001`, `010`	+1	+M
`011`	+2	+2M
`100`	−2	−2M
`101`, `110`	−1	−M

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Where do the radix-4 digits come from?

Combine two adjacent radix-2 terms while retaining the lower position's weight:

$$\begin{aligned} c_j &= d_{2j} + 2d_{2j+1} \\ &= (q_{2j-1} - q_{2j}) + 2(q_{2j} - q_{2j+1}) \\ &= -2q_{2j+1} + q_{2j} + q_{2j-1}. \end{aligned}$$

Thus $Q = \sum_j c_j 4^j$. Binary input bits restrict each digit to −2, −1, 0, 1, or 2. For example, `011` gives 0+1+1=2; `100` gives −2+0+0=−2; `111` gives −2+1+1=0.

Each higher group multiplies the base weight by four. Append a zero history bit to the lowest group; later groups overlap at the preceding group's top bit. For `001111`, read `110`, `111`, `001` from low to high, giving digits −1, 0, 1 and value −1+0×4+1×16=15.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Radix-4 example: −7 times 15

Use six-bit inputs $M = -7, Q =$`001111`. Form full twelve-bit signed partial products: extend the sign before shifting by the appropriate 4^j.

j	Bit positions	Three bits	c_j	$c_j M 4^j$	Twelve-bit partial product
0	$q_1\ q_0\ q_{-1}$	`110`	−1	+7	`000000000111`
1	$q_3\ q_2\ q_1$	`111`	0	0	`000000000000`
2	$q_5\ q_4\ q_3$	`001`	+1	−112	`111110010000`

Since $15 = -1 + 0 \cdot 4 + 1 \cdot 16$, the product is $7 - 112 = -105$, or `111110010111`. Addition uses a twelve-bit container; a carry beyond that width is outside the result, whose signed value is still -105.

Recheck internal width for radix-4. With the most negative multiplicand, $-2M = 2^n$ needs $n + 2$ signed bits. This section forms partial products in $2n$ bits; it does not claim that the radix-2 $n + 1$-bit accumulator directly supports every radix-4 design.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Faster multiplication starts with aligned partial products

Use unsigned four-bit inputs M=13 and Q=15. Generate each partial product in its actual bit position, using eight-bit rows throughout.

Row	Source	Eight bits	Unsigned value
P_0	$M \cdot q_0$	00001101	13
P_1	$M \cdot q_1 \cdot 2$	00011010	26
P_2	$M \cdot q_2 \cdot 4$	00110100	52
P_3	$M \cdot q_3 \cdot 8$	01101000	104

The target is their sum, 195. Repeatedly adding two rows into a conventional binary number can require a full carry propagation each time. Carry-save instead keeps two rows during reduction and converts them into a single result only at the end.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Three equal-weight bits become two output bits

At position j, x_j,y_j,z_j each have weight 2^j. A full adder reads these three bits and produces a same-position bit s_j and a higher-weight bit k_j:

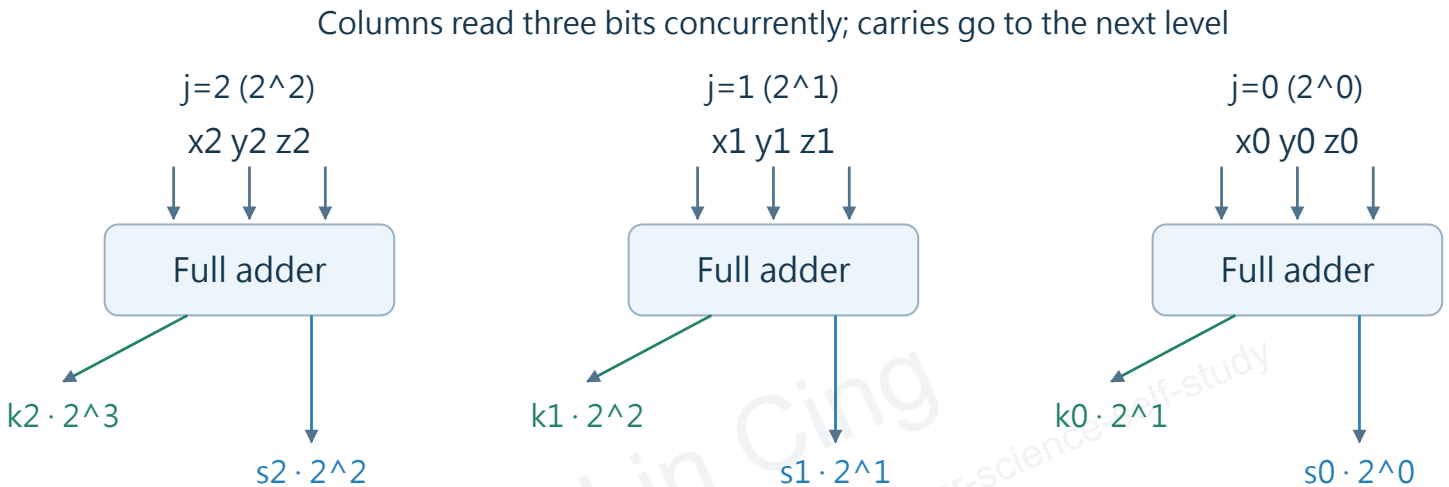
$$s_j = x_j \oplus y_j \oplus z_j, \qquad k_j = (x_j y_j) \vee (y_j z_j) \vee (z_j x_j).$$

$x_j+y_j+z_j$	Input patterns	s_j	k_j	Identity
0	000	0	0	0=0+2 • 0
1	001,010,100	1	0	1=1+2 • 0
2	011,101,110	0	1	2=0+2 • 1
3	111	1	1	3=1+2 • 1

Multiply each column identity by 2^j and sum: X+Y+Z=S+C, with $S = \sum_j s_j 2^j$ and $C = \sum_j k_j 2^{j+1}$. C already includes the one-bit left shift; do not shift it again. Products of bits denote AND; $\vee$ denotes OR.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A CSA level does not wait for a carry from its neighbor

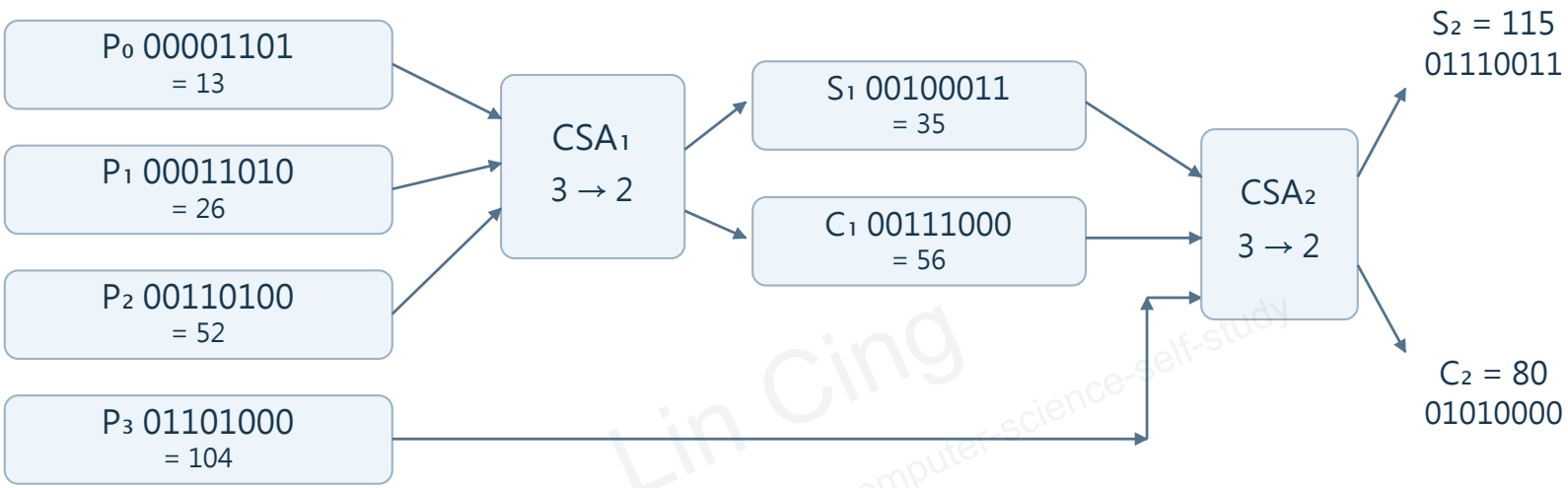


Blue: S row. Green: shifted C row. No carry link between same-level adders.

Each full adder reads the three current-level rows. Its carry goes into the next-level C row, not into its current-level neighbor. The columns evaluate concurrently once their inputs arrive.

The carry remains as data: only S+C represents the total. Adding these rows immediately at each level would reintroduce full carry propagation.

13×15: two reduction levels, then one ordinary addition



CSA_1 replaces P_0,P_1,P_2 with 35+56; P_3=104 bypasses it. CSA_2 replaces these three rows with 115+80. Each level preserves 195. C_1 and C_2 already include their left shifts; reduction neither divides the total nor discards a carry. The final ordinary addition still remains.

The final addition still has a carry dependency

Now add $01110011_2 + 01010000_2$. Let c_j be the carry entering position j, starting with c_0=0:

Position j, low to high	0	1	2	3	4	5	6	7
$S_2[j]$	1	1	0	0	1	1	1	0
$C_2[j]$	0	0	0	0	1	0	1	0
Carry-in c_j	0	0	0	0	0	1	1	1
Result bit	1	1	0	0	0	0	1	1
Carry-out c_{j+1}	0	0	0	0	1	1	1	0

Read result bits from high to low: $11000011_2 = 195$. The carry from position 4 affects positions through 7. A carry-propagate adder (CPA) resolves this dependency, using a ripple or faster prefix structure. Its delay is not zero.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

When may a high carry be discarded?

For three arbitrary unsigned W-bit inputs, S needs W bits, C may need W+1, and the exact total may need W+2. With W=4 and X=Y=Z=15, S=15 and C=30, so the exact total is 45.

If the specified result is only the low W bits, then

$$X + Y + Z \equiv S + (C \bmod 2^W) \pmod{2^W}.$$

Here truncating C to four bits gives 14. Then 15+14=29, whose low four bits are 13, equal to 45 mod 16. A correct modular result does not imply that the exact sum fits.

Unsigned n×n multiplication has an additional bound: all partial products are nonnegative and their total is below 2^{2n}. Every nonnegative row at every exact reduction level is at most this total, so 2n bits suffice. Booth rows require the declared two's-complement width and sign extension instead. For the earlier twelve-bit example, 7+0+3984=3991 represents −105 in twelve bits. Do not reuse the nonnegative unsigned-partial-product argument for the original signed values.

Reduction depth is not a count of full-width additions

For a model that groups each three rows into two and passes leftover rows directly to the next level,

$$R_{\ell+1} = 2\lfloor R_\ell/3 \rfloor + (R_\ell \bmod 3).$$

Starting with 32 rows gives 22,15,10,7,5,4,3,2: eight reduction levels. This exact count belongs to the declared grouping model. Actual Wallace/Dadda bit heaps schedule full and half adders column by column.

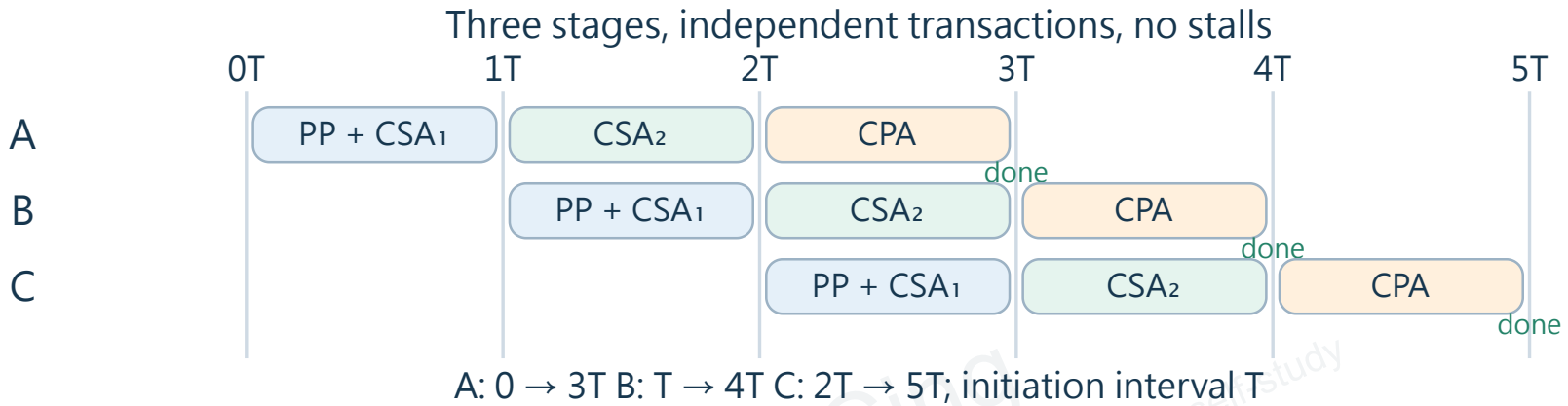
If t_CSA bounds each reduction level, an upper-bound model is

$$T \le t_{\mathrm{PP}} + 8t_{\mathrm{CSA}} + t_{\mathrm{CPA}} + t_{\mathrm{wire}}.$$

t_PP covers partial-product generation; t_wire covers additional wiring delay. Eight CSA levels are not eight W-bit CPAs. A balanced tree of ordinary two-input adders has only five levels for 32 rows, but each level may propagate carries. Comparing 8 with 5 alone says nothing about which circuit is faster.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pipeline initiation interval and result latency differ



Stage 1 generates partial products and performs CSA_1; stage 2 performs CSA_2; stage 3 performs the CPA. Each stage meets period T. Assume independent transactions and no stalls.

A enters at edge 0 and completes at edge 3; B enters at 1 and completes at 4. Latency is 3T, initiation interval T; N≥1 transactions finish at $(N+2)T$.

If B consumes A's product, it cannot enter at edge 1. T includes the slowest stage, register and clock costs; these timings alone do not establish speedup over an unpipelined circuit.

Traditional MIPS multiply instructions

`mult` and `multu` write a 64-bit product into HI and LO. `mfhi` and `mflo` move the halves into general-purpose registers.

Signed and unsigned hardware can share components, but upper product bits have different mathematical interpretations. Traditional `mult` and `multu` do not trap merely because the result exceeds 32 bits.

Taking only LO gives a low modular product. Exactness requires checking the full result against the intended result range.

`multu` interprets input bits as unsigned values. It is not an assertion about source-language types.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.4 Division starts with a quotient and remainder convention

For an unsigned dividend D and nonzero divisor M, require

$$D = QM + R, \qquad 0 \le R < M.$$

The identity and remainder bound together determine a unique answer. For example, 13=2・5+3 and 13=1・5+8 both satisfy the identity, but only the first satisfies the bound.

First use restoring division for unsigned inputs. Signed division additionally needs a rounding convention. Division by zero and the most negative signed input divided by −1 need separate handling.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bring in one input bit, then try one subtraction

If the current remainder is A and the next dividend bit is d, form $T = 2A + d$ and try $T - M$.

Trial	New quotient bit b	New remainder A′
$T - M \geq 0$	1	$T - M$
$T - M < 0$	0	T; add M back if subtraction has already occurred

Because $0 \leq A < M$ and $d \in \{0, 1\}$, we have $0 \leq T < 2M$. A single subtraction suffices: either branch restores $0 \leq A' < M$. This bound explains why each round emits just one quotient bit.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

13 divided by 5: prefix, quotient, and remainder

Read D=1101 from left to right. The quotient column below contains only generated bits, not the entire combined hardware register Q.

k	New d	Prefix P_k	Trial T	T−5	Generated quotient	Remainder
0	—	0	—	—	empty	0
1	1	1	1	−4	0	1
2	1	3	3	−2	00	3
3	0	6	6	1	001	1
4	1	13	3	−2	0010	3

Every row satisfies $P_k = q_k M + A_k$. Row 3 gives 6=1 • 5+1; the final row gives 13=2 • 5+3 with 3<5.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Q mixes unread input and quotient bits

Initialize A=`00000`, Q=`1101`, M=`00101`. Each left shift makes room for one quotient bit.

Round k	Remainder A (5 bits)		Q: unread input → quotient bits	q_k	$P_k=q_k\cdot5+A$
0	00000	= 0	1 1 0 1	0	0
1	00001	= 1	1 0 1 | 0	0	1
2	00011	= 3	0 1 | 0 0	0	3
3	00001	= 1	1 | 0 0 1	1	6
4	00011	= 3	0 0 1 0	2	13

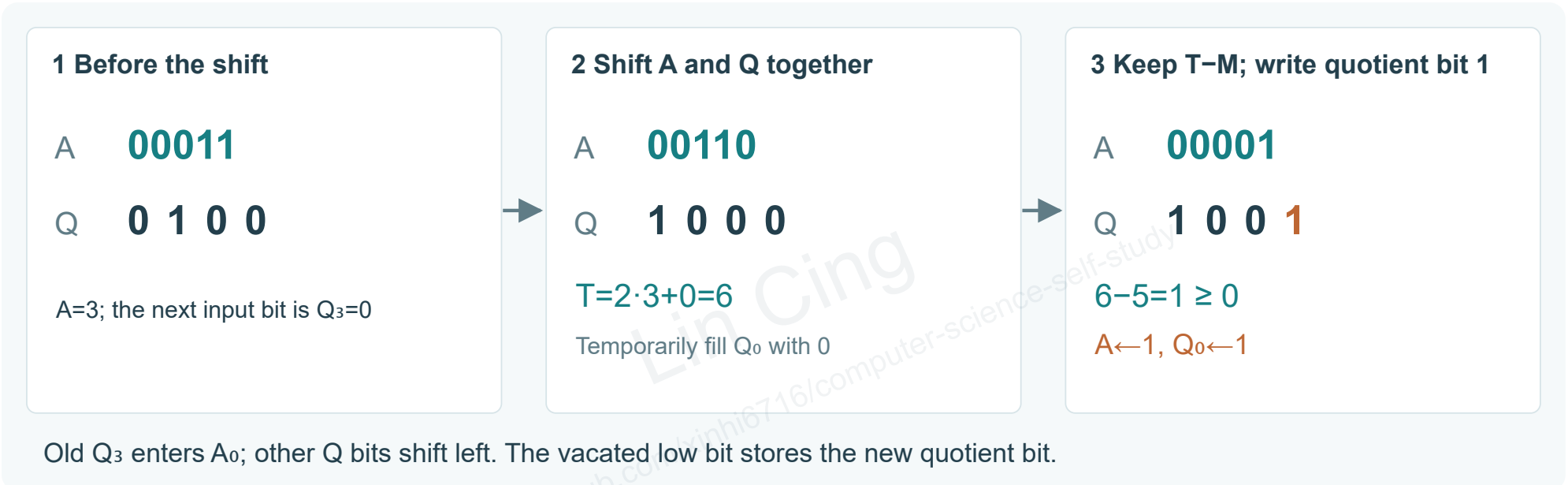
Teal: unread dividend bits Orange: quotient bits | Only at k=4 does the whole Q equal quotient 2.

Only the orange portion is the generated quotient q_k; the teal portion is unread input. The whole Q equals the quotient only after round n.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Round 3: shift across the register boundary before writing the bit

After round 2, A=3 and Q=`0100`. The next dividend bit is 0, so shifting forms T=6; subtract M=5.



The trial is +1, so retain A=`00001` and set $Q_0=1$, giving Q=`1001`. This Q is not a quotient of 9: its high 1 is still the last unread dividend bit.

Prove the prefix invariant and the remainder bound

Initially $P_0 = q_0 = A_0 = 0$. Suppose $P_k = q_k M + A_k$ and $0 \le A_k < M$. For the next bit d, let $T = 2A_k + d$, with b=1 exactly when T≥M. Then

$$P_{k+1} = 2P_k + d = (2q_k + b)M + (T - bM).$$

Thus $q_{k+1} = 2q_k + b$ and $A_{k+1} = T - bM$; T<2M preserves the remainder bound. After n rounds the prefix is D, proving the required identity and bound.

This implementation uses n+1 signed bits for T and the trial difference. The consumed prefix has at most n bits, and T is no larger than that prefix, so $T \le 2^n - 1$. Since $M \le 2^n - 1$, the difference is also within $[-(2^n - 1), 2^n - 1]$. The weaker bound T<2M alone would not justify the same signed width for arbitrary T.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Signed division truncates the quotient toward zero here

Divide magnitudes in unsigned containers, then restore signs. A nonzero remainder takes the dividend's sign.

D	M	Quotient Q	Remainder R	Check
7	2	3	1	7=3 • 2+1
−7	2	−3	−1	−7=(−3) • 2−1
7	−2	−3	1	7=(−3) • (−2)+1
−7	−2	3	−1	−7=3 • (−2)−1

The bound is $|R| < |M|$, not necessarily R≥0. With floor division, −7/2 instead gives quotient −4 and remainder +1. The identity still holds; the rounding convention differs. A zero remainder needs no sign assignment.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Boundary cases and timing require separate checks

The magnitude 2^{n-1} of the most negative signed input fits in n unsigned bits but not in n positive signed bits. Dividing it by −1 produces a positive quotient beyond the signed upper bound.

Traditional MIPS32 `div` and `divu` place quotient in LO and remainder in HI. In this chapter's version, zero division and signed quotient overflow do not automatically cause an arithmetic trap; software must check these inputs rather than depend on HI/LO contents.

The combined-register algorithm here consumes n bits in n rounds. The book's initial wide-divisor algorithm uses n+1 iterations. Do not mix their loop counts. SRT designs may choose multiple quotient bits per round, but selection, normalization, and correction also cost time; two bits per round alone does not prove twice the speed.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Arithmetic right shift and signed division can disagree

An arithmetic right shift rounds a two's-complement input toward negative infinity; the signed division convention here rounds toward zero.

Operation on −7	Result	Rule
Arithmetic right shift by one	−4	$\lfloor -7/2 \rfloor$
Signed division by two	−3	Truncate toward zero
Add one, then arithmetic shift	−3	$\lfloor (-7+1)/2 \rfloor$

For division by 2^k, negative inputs may first receive bias $2^k - 1$, then shift arithmetically. Specify $1 \leq k < n$ and an overflow-safe typed implementation. Reciprocal-multiply replacements must likewise preserve the original quotient rule.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.5 IEEE 754 representation

$$value = (-1)^s \, significand \, 2^e.$$

Binary32 has 1 sign bit, 8 exponent bits, and 23 fraction bits. Normal values use exponent fields 1 through 254, bias 127, and an implicit leading 1.

Binary64 has fields of 1, 11, and 52 bits, with bias 1023. Exponent range and significand precision solve different representation problems.

"Floating point" alone does not specify a format.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Floating-point fields: precision includes an implicit bit

IEEE 754 binary32 fields and encoding of -12.5



0xC1480000

sign=1; exponent=10000010₂

-12.5 = -1.1001₂ × 2³

value = (-1)^s × 1.fraction × 2^(exponent-127)

Zero exponent uses subnormal rules. All-one exponent denotes infinity or NaN; hidden 1 is not universal.

For normal values, $(-1)^s (1.F)_2 \, 2^{E-\text{bias}}$. Binary32 has p=24, binary64 p=53. Subnormals use a leading 0; binary32 uses exponent −126. The hidden 1 does not apply when E=0.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Special values and flags

An all-ones exponent with zero fraction denotes signed infinity. A nonzero fraction denotes NaN. Quiet and signaling NaNs have distinct invalid-operation behavior and payload rules.

IEEE exception classes include overflow, underflow, inexact, invalid, and division by zero. A flag and a language exception are not the same mechanism. Trap controls determine whether ordinary nontrapping continuation occurs.

$+\infty + (-\infty)$ is invalid and ordinarily returns quiet NaN. $1/(+\infty) = +0$ under the usual IEEE rules.

Separate the numerical result, status flag, and enabled trap.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Floating-point addition: alignment precedes rounding

For finite normal inputs, compare exponents and shift the smaller significand to a common scale. Opposite signs require magnitude subtraction; the larger magnitude determines the sign. Information shifted beyond retained precision must survive until rounding.

Normalize the sum or difference, round to the target precision, and renormalize if rounding carries out. Special values, zeros, overflow, and subnormal results need their own paths.

The following reduced model has **p=4 significant bits including the leading 1, RNE, and an unbounded exponent range**. It exposes every intermediate bit; it is not binary32.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

An alignment trace where sticky changes the answer

Let a=1 and b=9/128=0.0703125. Both are exact at p=4. Aligning $b=1.001_2\times2^{-4}$ gives $0.0001001_2\times2^0$.

p=4 including the leading 1; RNE; unbounded exponent range

$\times2^0$	Retain 4 bits	G	R	Tail bits
a = 1	1.000	0	0	00
b = 9/128	0.000	1	0	01
a + b	**1.000**	**1**	**0**	**01**

$b = 1.001_2 \times 2^{-4} = 0.0001001_2 \times 2^0$

S = OR(0,1) = 1 → GRS = 101 → 1.001_2 = 1.125

Replacing the tail by 00 would create a false tie and return 1.000_2.

The exact sum is 137/128. Its adjacent candidates are 1 and 9/8, with midpoint 17/16=136/128. RNE selects 9/8 with error 7/128. Returning 1 after discarding the tail would give error 9/128, exceeding half an ULP=1/16.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Cancellation: find the leading 1 again

At p=4, compute exact 0.75−0.6875. After subtraction, shift left three positions.

p=4, RNE: 0.75 − 0.6875; both inputs are exact

a	**1.100 000**	× 2^{-1}
− b	**1.011 000**	× 2^{-1}
Subtract	**0.001 000**	× 2^{-1}
Shift left 3	**1.000 000**	× 2^{-4}

Each left shift decreases the exponent by one; value is unchanged.

GRS=000; the result 1/16=0.0625 is exact, so inexact=0.

The difference is exact. Cancellation can amplify existing input error without creating rounding error. At the minimum exponent, use subnormals.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Derive nearest-even from G, R, and S

Work on the result's magnitude. L is the least retained bit, G the next bit, and R the following bit. S is the OR of every remaining bit, summarizing whether the tail is nonzero.

In retained-ULP units, G has weight 1/2, R has weight 1/4, and the tail is below 1/4. G=0 is below midpoint. G=1,R=S=0 is exactly halfway. G=1 with R or S nonzero is above halfway. At a tie, choose the candidate with even retained encoding. Thus

$$\text{inc} = G \wedge (R \vee S \vee L).$$

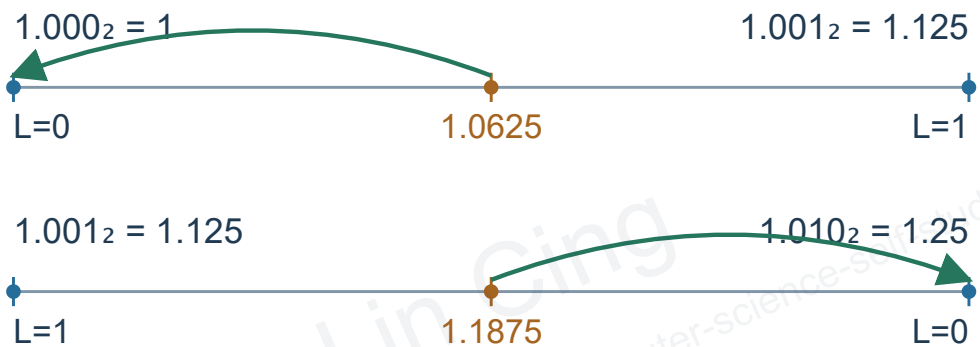
If inc=1, increment the retained **magnitude** by one ULP, then restore the sign. For a negative result this can make the number more negative; it is not always rounding toward +∞. Any nonzero discarded part implies inexact.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Two midpoints, two parity decisions

Even refers to the significand encoding: the intervals below select opposite endpoints.

p=4, RNE; each number line shows one adjacent interval



inc = G ∧ (R ∨ S ∨ L)

Round the magnitude, then restore the sign: −1.1875 → −1.25.

At p=4, 1.1111_2 has L=1,GRS=100. Increment: $1.111_2+0.001_2=10.000_2=1.000_2\times2^1$. Renormalization increases the exponent.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Two binary32 rounding checkpoints

Binary32 has p=24 and spacing 2^{-23} on [1,2). For $1+2^{-24}$, L=0 and GRS=100 select 1 (0x3f800000). For $1+3 \cdot 2^{-25}$, GRS=110 selects $1+2^{-23}$ (0x3f800001). Both operations set inexact.

Binary32 round-to-nearest-even and five FP exceptions

Tie: $1+2^{-24}$
GRS=100, retained LSB=0
candidate: 1.0
inexact

Above half: $1+3 \cdot 2^{-25}$
GRS=110
round up: $1+2^{-23}$
inexact

Underflow tie: 2^{-150}
midpoint: 0 and 2^{-149}
even→+0
underflow + inexact

Overflow: max+max
exact result > max finite
rounded→+∞
overflow + inexact

Invalid: 0×∞, ∞−∞, 0/0 → quiet NaN; 1/±0 → signed infinity.

G/R/S = guard/round/sticky. Ties select retained LSB zero; NaN follows payload and comparison rules.

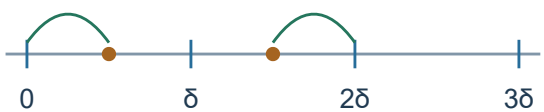
Here 2^{-24} is half the adjacent-value spacing. FMA rounds exact ab+c once, whereas separate multiply and add have two rounding points. A later worked example compares their output bits.

Subnormal representation and underflow flags are different questions

Put $\delta = 2^{-149}$. Positive binary32 subnormals are $F\delta$ for fraction integers $1 \le F \le 2^{23}-1$. The smallest normal is $2^{23}\delta = 2^{-126}$. Spacing remains δ across this boundary.

binary32: $\delta = 2^{-149}$; RNE; gradual underflow

Near zero



$\delta/2 \to 0$; $3\delta/2 \to 2\delta$

underflow=1, inexact=1

Exact δ: both flags are 0

Near the smallest normal



0x007fffff = $(2^{23}-1)\delta$

0x00800000 = $2^{23}\delta$

Spacing across the boundary is still δ

The right zoom starts at index $2^{23}-2$; the panels do not share an origin.

Subnormal does not imply an underflow flag. Exact $2^{-126}/2$ gives 0x00400000 without error; the two inexact midpoint cases set both flags. The next page specifies the environment.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Check exactness at the subnormal boundary

Fix RNE, gradual underflow, disabled traps, initially clear flags, and tininess after rounding. Every input is exact in binary32. UF and NX denote underflow and inexact.

Operation	Exact value	RNE output bits	UF	NX
$2^{-126} / 2$	2^{-127}	0x00400000	0	0
$\delta / 2$	$\delta/2$	0x00000000	1	1
$(3\delta) / 2$	$3\delta/2$	0x00000002	1	1
Smallest normal − largest subnormal	δ	0x00000001	0	0

At the midpoints, even fraction integers win: choose 0 between 0 and 1, and 2 between 1 and 2. Flags record conditions, not error magnitudes. Flush-to-zero or treating subnormal inputs as zero changes this model.

See Oracle Numerical Computation Guide § 2.3 for representation and gradual underflow.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Special values: report the result and the exception

Maximum finite plus itself returns $+\infty$ under RNE, setting overflow and inexact. The invalid operations $0 \times \infty$, $\infty - \infty$, and $0/0$ return quiet NaN when traps are disabled.

Finite nonzero divided by signed zero returns infinity with the XOR of the input signs, setting divide-by-zero. Infinity divided by +0 remains infinity without that flag. See the Oracle Numerical Computation Guide exception table.

NaN comparison depends on the predicate: ordinary equality is false even for a NaN compared with itself; inequality is true. Distinguish ordered and unordered predicates. +0 and −0 compare equal, but reciprocal and signbit distinguish them.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Floating-point multiplication

For normal inputs, multiply $(1.F_a)(1.F_b)$, add unbiased exponents, and XOR signs. The exact significand product lies in $[1, 4)$. Normalize and round it to target precision.

Subnormal inputs may require normalization before multiplication. A tiny result can enter the gradual-underflow path. Special combinations are handled first. In particular, $0 \times \infty$ is invalid.

Multiplication avoids exponent alignment but needs a wide significand product and final rounding.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

MIPS floating-point instructions

The book's model uses coprocessor-1 registers. `lwc1` and `swc1` move words. `add.s` and `add.d` select single and double formats.

Comparisons set condition information examined by instructions such as `bc1t` and `bc1f`, with the traditional control-transfer delay slot. NaNs require ordered and unordered comparison semantics.

Floating-point and general-purpose register spaces differ. The compiler must respect register format and saved-area requirements.

These encodings and register rules are not interchangeable with ARM or RISC-V.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Accuracy and rounding modes

Nearest-even is a common default. Directed modes round toward positive infinity, negative infinity, or zero. Directed rounding can support interval bounds.

An ULP is the spacing between adjacent representable values in the relevant binade. Small absolute error does not imply small relative error. Spacing grows with exponent.

Significand width controls precision. Exponent width controls range. Adding exponent bits does not add significand accuracy.

Compare binary32 and binary64 spacing at the same value rather than comparing their largest values alone.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NaNs, signed zero, and comparisons

NaN is unequal to every value under ordinary equality, including itself. Use `isnan` or an unordered comparison to detect it.

IEEE equality treats positive and negative zero as equal, but reciprocals, sign-copy operations, and serialization can expose their different sign bits.

$\infty - \infty$, $0 \times \infty$, and ∞ / ∞ are invalid. Signed integer ordering of raw bits cannot implement ordinary floating-point ordering across negative values and NaNs.

A comparison algorithm must specify its treatment of NaNs and signed zeros.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.6 Parallelism and nonassociativity

Integer addition remains associative modulo a fixed power of two. Floating-point rounding can instead make

$$(a + b) + c \neq a + (b + c).$$

A changed reduction tree changes rounding points. Different answers can therefore be a numerical reproducibility issue without any data race. Bitwise reproducibility requires a fixed order and matching formats, rounding, intermediate precision, and FMA contraction, or an accumulator with an explicit order-independent guarantee. Compensation and higher precision improve error but do not alone guarantee order independence.

For standard Kahan summation with each operation rounded to binary32 RNE, `[2^25,1,-2^25]` gives 0, while `[2^25,-2^25,1]` gives 1. In the first order, adding 1 leaves the sum at 2^{25} and compensation at -1. Then $-2^{25} + 1$ is a midpoint and rounds back to -2^{25}, producing zero. The second order cancels exactly before adding 1. Compensation has not removed dependence on order or environment.

Changing grouping changes rounding points

At $x = 2^{24}$, the binary32 ULP is 2. `x+1` is the midpoint between x and x+2, and the even candidate is x:

$$(x + 1) - x = 0, \qquad (x - x) + 1 = 1.$$

These expressions agree over exact reals but differ under per-operation rounding.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

FMA rounds the exact multiply-add only once

Let $a = b = 1 + 2^{-23}$, bits `0x3f800001`, and $c = -(1 + 2^{-22})$, bits `0xbf800002`.

$$ab = (1 + 2^{-23})^2 = 1 + 2^{-22} + 2^{-46}.$$

Separate multiply and add first round the product to $1 + 2^{-22}$, then cancel with c to +0. A fused operation computes exact $ab + c = 2^{-46}$ before its sole rounding, yielding `0x28800000`. It changes the intermediate rounding, not real algebra. The old MIPS core does not guarantee FMA; an extension must specify format and rounding mode.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.7 The x86 floating-point case

The book compares x87 extended-precision stack registers with SSE2 XMM scalar and packed operations. Keeping an intermediate wider than its memory format can differ from rounding every operation to that format.

ABI rules, compiler flags, spills, and precision controls affect the rounding points. SIMD operations also require correct lane and alignment semantics.

Do not silently substitute modern AVX behavior for a historical x87 example. State the actual arithmetic environment.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.8 Arithmetic fallacies

Approximation has exact rules. Applications can depend on rounding and exception flags.

Fast-math transformations can permit reassociation or assumptions about NaNs and signed zero. They can change reproducibility and numerical behavior.

There is no universal fixed ratio between divide and multiply costs. Operand dependence, pipelines, specialization, and instruction mix all matter.

Pair each performance claim with a specified operation and timing model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.9 Arithmetic synthesis

Integer width, overflow, and modular behavior must be specified together. Multiply/divide iteration count does not by itself determine latency or throughput.

A floating-point unit implements representation, arithmetic, and rounding with exceptions. Addition emphasizes alignment and lost-bit information. Multiplication emphasizes the significand product and normalization.

Verification needs normal values, zeros, subnormals, infinities, NaNs, overflow, underflow, and halfway cases.

Ordinary exact sums alone do not exercise the rounding contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Numerical history

Earlier machines used incompatible exponent widths, biases, and representations. IEEE 754 standardized formats, rounding, and special values to make numerical behavior more predictable.

Hardware evolved through iterative arithmetic, recoding, compression trees, pipelined floating point, and SIMD. This did not eliminate numerical analysis. Ill-conditioning, large dynamic range, and reproducible parallelism remain algorithmic concerns.

Separate format, arithmetic hardware, rounding, and numerical algorithm.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

End-of-chapter check

Determine signed and unsigned overflow, trace small multiply and divide states, decode binary32 special cases, and justify guard/round/sticky decisions. Give a counterexample to floating-point associativity.

Use exercises and solutions separately. Show representation and state transitions, not only the final decimal number.

A bit-level trace makes both the arithmetic policy and its boundary cases inspectable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study